COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 28 — Project Integration & End-to-End Practical

---

Section 11: End-to-End Team Activity — Project Synthesis Packet

Comprehensive integration packet bringing together requirements, architectural design, Jira task planning, Git version control strategy, test verification, and deployment handover for CEMS.

1. Core User Story & Acceptance Criteria

- **User Story:** "As an authenticated college student, I want to securely register for an open technical workshop through the portal, so that my attendance seat is guaranteed and verified via a mobile QR ticket."
- **AC-1:** Registration form validates student eligibility and rejects already registered students.
- **AC-2:** Database transaction uses row-level locking to atomically decrement available seat count.
- **AC-3:** System generates unique 8-character confirmation code and dynamic QR ticket immediately.
- **AC-4:** Confirmation email job is dispatched to asynchronous queue within 2 seconds of commit.

2. System Architecture & Relational Entities

Architecture Topology: React.js SPA (Frontend) communicates via HTTPS REST APIs with Express.js Backend Application Cluster, backed by PostgreSQL 16 relational database and Redis 7 asynchronous job queue.
Data Entities: `User` (userId, name, email, rollNo, role); `Event` (eventId, title, maxCapacity, currentCount, status); `Registration` (registrationId, studentId, eventId, qrToken, status, createdAt).
Foreign Keys: `Registration.studentId -> User.userId`, `Registration.eventId -> Event.eventId`.

3. Git Branching Strategy & Commit Standards

- **Branching Model:** GitFlow. `main` (production-ready) <- `develop` (integration) <- `feature/CEMS-102-atomic-booking`.
- **Commit Message 1:** `feat(booking): implement atomic seat reservation with SELECT FOR UPDATE [CEMS-102]`
- **Commit Message 2:** `test(booking): add concurrency integration test for simultaneous last-seat claims [CEMS-102]`

4. Test Suite Matrix

Test Case ID	Test Scenario	Precondition	Expected Outcome
TC-E2E-01	Single Student Happy Path	Event capacity 50/100	Seat allocated; count becomes 51; QR ticket rendered.
TC-E2E-02	Exact Boundary	Event capacity 99/100	100th seat allocated;

	Registration		status badge toggles to "FULL".
TC-E2E-03	Concurrent Race Condition	Event capacity 99/100 (1 seat left)	Simultaneous hits: 1 succeeds (200 OK); 1 fails (409 Conflict).
TC-E2E-04	Duplicate Booking Attempt	Student already registered	Request blocked with validation error; count unchanged.

Section 14: Homework — End-to-End Requirement Traceability

Tracing a single functional requirement through design, tasks, code, testing, and release notes for a Hospital Appointment Booking System.

- **1. Requirement:** REQ-HOSP-04: "Patients shall be able to book an available consultation slot with a specialist doctor."
- **2. Architectural Design:** Mapped to `DoctorScheduleModule` and `AppointmentBookingService`; relational table `appointments` with unique constraint `(doctor_id, appointment_timestamp)`.
- **3. Implementation Tasks:** Jira Tasks: HOSP-211 (Database schema migration); HOSP-212 (Doctor slot calendar REST API); HOSP-213 (Frontend slot picker widget).
- **4. Verification Evidence:** Automated unit test `DoctorBookingTest.java` (asserts slot locking) and Cypress E2E test `spec-booking.cy.ts` passing in CI build #114.
- **5. Release Note:** "Release v1.1.0: Added online appointment scheduling for Cardiology and Pediatrics clinics with real-time calendar slot validation."
- **6. Reflection on Disconnected Records:** When requirements, code commits, and test cases are maintained in disconnected silos, a developer might modify a booking validation rule to resolve a minor bug without realizing they violated an unlinked medical compliance requirement, introducing undetected legal liabilities into production.

DRAFT